certora

Security Assessment Report

Compound Multiplier PR-32

March 2026

Table of Contents

Project Summary 4
- Project Scope 4
- Project Overview 4
- Protocol Overview 4

Assessment Methodology 6

Threat and Security Overview 7
- System Overview 7
  - Core Operations: 7
  - Plugin Architecture: 7
- Assets 8
- Actors 8
- Trust Assumptions 9
- Attack Vectors 9
- Findings Summary 11
- Severity Matrix 11

Detailed Findings 12
- High-Severity Issues 14
  - H-01: `stETH.submit()` Returns Shares, Not Token Amount — Partial stETH Stranded in Contract 14
- Medium Severity Issues 16
  - M-01: multiply() and adjust() use incorrect leverage checks, weakening maxHealthFactorDrop protection 16
- Low Severity Issues 18
  - L-01: OKX Plugin _extractFromTokenCommissionRate can read calldata at out-of-bounds offsets for multi-referrer commission layouts 18
  - L-02: Swap adapters do not enforce full amountIn consumption, so partial fills finalize as if the whole operation executed 19
  - L-03: Front-runnable allowBySig enables denial of service (DoS) on user transactions 20
  - L-04: Adjusted event reverts on underflow when user has base supply 21
- Informational Severity Issues 23
  - I-01: Swap plugins and external calls fail to reset approvals, accumulating residual allowances 23
  - I-02: adjust() is functionally redundant with multiply() and can be merged 24
  - I-03: UniswapV3Plugin rejects zero-fee flash loans unnecessarily 25
  - I-04: cover() function's strict loanDebt <= borrowBalance check prevents fully closing positions due to interest accrual 26
  - I-05: _multiply() does not validate collateralAmount > 0 for ERC-20 path 27
  - I-06: _exchange() does not validate fromAmount against user's collateral balance 28
  - I-07: Missing opts.comet validation allows arbitrary contract execution 29
  - I-08: Unvalidated swap routing parameters allow token diversion 30
  - I-09: cover() lacks deleveraging invariant, allowing transactions to silently worsen account safety 31

I-10: Zero-value plugin selectors can be registered even though they are unusable at runtime..... 32
I-11: _dust() NatSpec no longer matches the new dust-handling logic..... 33
Disclaimer..... 34
About Certora..... 34

Project Summary

Project Scope

Project Name	Repository Link	Initial Commit Hash	Final Commit Hash	Platform
Compound Multiplier PR-32	https://github.com/woof-software/compound-multiplier	d8b83e1	8897967	EVM

Project Overview

This document describes the security review of **Compound Multiplier PR-32**. The work was undertaken from **March 23rd, 2026** to **March 27th, 2026**.

The following contract list is included in our scope:

- contracts/plugins/swap/OKXPlugin.sol
- contracts/CometFoundation.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Compound is a decentralized lending protocol where users supply assets, use supported collateral to borrow the market's base asset, and maintain positions subject to collateralization limits. Compound Multiplier is an adapter around Compound V3 Comet that lets users atomically leverage, deleverage, and rebalance positions through flash-loan and swap plugins.

PR-32 introduces a new adjust() flow for increasing leverage on an existing position, while effectively treating deleveraging as the existing cover() path. It also changes cover() to accept caller-supplied loanDebt instead of deriving the flash-loan amount on-chain, and updates the

shared leverage-bound logic through the _maxLeverage() / maxHealthFactorDrop changes. In addition, the PR modifies the OKX swap plugin to account for appended from-token commission metadata when validating swap quotes.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System Overview

CometFoundation is a single smart contract that enables atomic leveraged position management on Compound V3 (Comet) markets. It bridges flash loan providers with DEX swap aggregators so users can create, increase, reduce, or rebalance leveraged positions in a single transaction.

Core Operations:

- **Multiply** — deposit collateral, flash-borrow base asset, swap to more collateral, supply to Comet, and borrow against it to repay the flash loan. Result: a leveraged position.
- **Adjust** — increase leverage on an existing position without depositing new capital. Flash-borrows additional base asset, swaps to collateral, supplies it, and borrows to repay.
- **Cover** — reduce or close a leveraged position. Flash-borrows base asset to repay part of the user's debt, withdraws collateral, swaps it back, and repays the flash loan.
- **Exchange** — swap one collateral asset for another within the user's Comet position while preserving the debt level.

Plugin Architecture:

The system is modular, with two plugin categories registered at deployment:

- **Flash Loan Plugins** — Morpho Blue, Euler V2, AAVE V3, Balancer, Uniswap V3
- **Swap Plugins** — 1inch V6, LiFi, OKX, wstETH (Lido wrapper)

Plugins are identified by keccak256(endpoint, callbackSelector) and invoked via delegatecall during flash loan callbacks. Intermediate state is stored using EIP-1153 transient storage for gas efficiency. The contract supports EIP-712 signature-based approvals (allowBySig) and native ETH deposits with automatic WETH wrapping.

Assets

- **User collateral tokens** (ERC-20s deposited into Comet markets — e.g., WETH, wstETH, cbETH)
- **Base asset / borrowed tokens** (USDC or other Comet base tokens)
- **Flash loaned funds** which are temporarily held by CometFoundation during multiply/cover/adjust/exchange
- **Residual token allowances** accumulated on the CometFoundation contract
- **Stuck/dust tokens** residing in the CometFoundation between operations
- **User health factor / borrow headroom** (safety margin that determines liquidation risk)
- **EIP-712 signature nonces** (consumable authorization for allowBySig)
- **Leverage position integrity** (ratio of debt to collateral per user)

Actors

- **End User** — calls multiply(), adjust(), cover(), exchange() to manage leveraged positions
- **Attacker / Mempool Observer** — front-runs allowBySig signatures, crafts malicious swap calldata, or deploys fake Comet contracts
- **Malicious / Compromised Frontend** — supplies crafted router calldata to divert tokens via unvalidated swap parameters
- **DEX Routers** (1inch, LiFi, OKX) — external swap execution; trusted with token approvals
- **Flash Loan Providers** (Uniswap V3, AAVE, Morpho) — supply flash liquidity; receive approval for repayment
- **Comet Market (Compound V3)** — lending protocol that enforces collateralization; opts.comet is caller-controlled
- **Contract Admin/Deployer** — configures plugins, selectors, and plugin registry at construction time

Trust Assumptions

- **Admin-managed configuration** is maintained correctly, including plugin registration and any approved integration lists.
- **Swap router calldata is honest** — the protocol forwards raw user-supplied calldata to routers, trusting that srcReceiver, executor, approveTo, callTo, and swap paths are benign
- **Routers fully consume approved amountIn** — no adapter verifies full spend; partial fills silently finalize
- **Pre-execution leverage checks approximate post-execution state** — the protocol assumes idealized math before execution equals realized state after fees, interest accrual, and rounding
- **Caller-supplied market and swap parameters** correspond to the intended external integrations.
- **opts.comet is a legitimate Compound V3 market** — no on-chain allowlist validates this
- **Users hold single-collateral positions** — _adjust() compares total debt against one collateral's capacity, breaking for multi-collateral accounts
- **borrowBalance is stable between off-chain calculation and on-chain execution** — interest accrual invalidates this, preventing full position closure
- **External protocols such as Compound V3 and supported swap/flash-loan integrations** are not malicious or compromised.
- **cover() always deleverages the user** — no invariant enforces that post-operation health improves

Attack Vectors

- **Fake Comet injection** — attacker passes a malicious contract as opts.comet; the foundation approves and supplies collateral to it
- **Swap calldata manipulation / token diversion** — compromised frontend crafts 1inch/LiFi/OKX payloads with attacker-controlled srcReceiver/executor/approveTo/callTo, siphoning approved tokens while returning minimum output

- **Signature front-running (DoS)** — attacker extracts allowBySig signature from mempool, consumes the nonce directly on Comet, causing the user's transaction to revert

- **Health factor bypass via pre-check divergence** — maxHealthFactorDrop is checked pre-execution with idealized inputs; actual post-execution state diverges due to existing debt, flash loan fees, and multi-collateral positions, pushing users closer to liquidation than specified

- **Inflated debt ceiling from flawed leverage math** — leveraged * _maxLeverage multiplies collateral value by a leverage ratio meant for equity, producing a massively inflated cap; maxHealthFactorDrop is increasing allowed leverage

- **Arithmetic underflow on event emission** — interest rounding causes postDebt - repaymentAmount to underflow, reverting adjust() transactions entirely

- **Partial fill exploitation** — router executes a partial swap; the protocol finalizes the operation treating full debt as incurred while only partial collateral was received

- **Dust debt from interest accrual** — cover() repays a stale loanDebt amount; by execution time, accrued interest leaves irremovable dust debt

- **Silent position degradation via cover()** — absence of a deleveraging invariant allows a crafted cover call (e.g., via malicious frontend) to worsen the user's health factor

- **Residual allowance accumulation** — routers/flash loan providers retain unconsumed allowances; if any provider is later compromised, all residual allowances are drainable

- **Out-of-bounds calldata read (OKX plugin)** — multi-referrer commission layouts with short payloads cause offset underflow, reading garbage values for totalRate

- **Zero-fee flash loan rejection** — UniswapV3Plugin unnecessarily blocks zero-fee flash loans

- **Gas griefing via zero-amount inputs** — _multiply() and _cover() accept zero amounts

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	1	1	1
Medium	1	1	1
Low	4	4	3
Informational	11	11	7
Total	17	17	12

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
H-01	`stETH.submit()` Returns Shares, Not Token Amount — Partial stETH Stranded in Contract	High	Fixed
M-01	multiply() and adjust() use incorrect leverage checks, weakening maxHealthFactorDrop protection	Medium	Fixed
L-01	OKX Plugin _extractFromTokenCommissionRate can read calldata at out-of-bounds offsets for multi-referrer commission layouts	Low	Fixed
L-02	Swap adapters do not enforce full amountIn consumption, so partial fills finalize as if the whole operation executed	Low	Acknowledged
L-03	Front-runnable allowBySig enables denial of service on user transactions	Low	Fixed
L-04	Adjusted event reverts on underflow when user has base supply	Low	Fixed
I-01	Swap plugins and external calls fail to reset approvals, accumulating residual allowances	Informational	Fixed
I-02	adjust() is functionally redundant with multiply() and can be merged	Informational	Fixed

I-03	UniswapV3Plugin rejects zero-fee flash loans unnecessarily	Informational	Fixed
I-04	cover() function's strict loanDebt <= borrowBalance check prevents fully closing positions due to interest accrual	Informational	Fixed
I-05	_multiply() does not validate collateralAmount > 0 for ERC-20 path	Informational	Fixed
I-06	_exchange() does not validate fromAmount against user's collateral balance	Informational	Fixed
I-07	Missing opts.comet validation allows arbitrary contract execution	Informational	Acknowledged
I-08	Unvalidated swap routing parameters allow token diversion	Informational	Acknowledged
I-09	cover() lacks deleveraging invariant, allowing transactions to silently worsen account safety	Informational	Acknowledged
I-10	Zero-value plugin selectors can be registered even though they are unusable at runtime	Informational	Acknowledged
I-11	_dust() NatSpec no longer matches the new dust-handling logic	Informational	Fixed

High-Severity Issues

H-01: `stETH.submit()` Returns Shares, Not Token Amount — Partial stETH Stranded in Contract

Severity: High	Impact: Medium	Likelihood: High
File: plugins/swap/WstEthPlugin.sol	Status: Fixed	

Description:

In `_lidoSwap()`, the return value of `IStEth(stEth).submit()` is used as the stETH token amount for the subsequent `approve` and `wrap` calls:

```javascript
uint256 stAmount = IStEth(stEth).submit{ value: amountIn }(address(this));
IERC20(stEth).safeIncreaseAllowance(wstEth, stAmount);
require(IWstEth(wstEth).wrap(stAmount) > 0, ICA.InvalidAmountOut());
```

However, Lido's submit() returns the number of **stETH shares** minted, not the **stETH** token amount. Because stETH is a rebasing token where 1 share < 1 stETH (and the gap grows over time as rewards accrue), **stAmount** underestimates the actual stETH balance received. This means:

- Line 65 approves fewer tokens than the contract actually holds.
- Line 66 wraps fewer stETH tokens than received.
- The difference (`actual stETH balance - shares`) remains stranded in the contract after each swap, representing a loss to the user.

Users receive less wstETH than they should on every swap. The magnitude of the loss equals the stETH rebase ratio minus 1 (currently ~11-12% and growing over time as Lido rewards accumulate). The stranded stETH accumulates in the contract and is not recoverable by the user.

Recommendations:

Use a balance-before/balance-after pattern to determine the actual stETH amount received:

```javascript
- uint256 stAmount = IStEth(stEth).submit{ value: amountIn }(address(this));
+ uint256 stBefore = IERC20(stEth).balanceOf(address(this));
+ IStEth(stEth).submit{ value: amountIn }(address(this));
+ uint256 stAmount = IERC20(stEth).balanceOf(address(this)) - stBefore;
```

Customer Response:

The team confirmed the issue through testing — stETH balance does not correspond to the shares value returned from `submit()`. The fix using the balance-delta pattern was applied in PR#34. The bug was not deployed to production.

Fix Review:

Fix looks good.

Medium Severity Issues

M-01: multiply() and adjust() use incorrect leverage checks, weakening maxHealthFactorDrop protection

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/CometFoundation.sol	Status: Fixed	

Description:

multiply() and adjust() both rely on pre-execution leverage checks to enforce maxHealthFactorDrop, but those checks do not consider the user's actual end-state health factor and as a result they do not provide the protection the parameter suggests.

- multiply() validates only the new leveraged amount from the freshly supplied collateral and borrowed amount. It ignores the user's existing account state, including prior debt, prior collateral, and other collateral assets already supporting the position.
- In `adjust()`, the check is incorrect because it uses the user's gross collateral as the base for `_maxLeverage()`, instead of the user's equity (collateral - debt). For example with $10,000 collateral, $8,000 debt, and CF = 80%, the user is already at the 5x leverage. The current formula still allows: 8,000 + debtDelta <= 10,000 * 5 which overstate available leverage by a lot. Also, the check is incorrect for multi-collateral accounts, since it compares the user's total debt against only the selected collateral.
- In both multiply()/adjust(), the precheck also is not taking into account the flash-loan fee, even though the realized debt increase is debt + fee, so the final position is always slightly worse.

As a result, the leverage checks are only approximate and maxHealthFactorDrop is therefore not enforced as a reliable protection. After all, users can end up closer to liquidation than the parameter suggests, or encounter late reverts.

Recommendations:

Consider reworking the leverage checks to validate the user's real final position instead of pre-execution approximation. In particular, multiply() and adjust() should take into account the full account state, including existing debt, all supporting collateral, and the eventual flash-loan fee. Also, in order for the maxHealthFactorDrop to act as protection, it should be enforced against the final post-operation position.

Customer Response:

Fixed in commit 99dd88c, 484e62f - *"Implemented a post check for health validation. Variable `maxHealthFactorDrop` was renamed into `healthBuffer` and now checks that there is a buffer between debt and collateral values before the liquidation.*

The validation takes into account the whole debt and the whole collateral values after the operation.

For now, checking collateral value against specified collateral only rather than all collaterals. We assume that users who use multiply won't supply any additional collateral. Checking all collateral might be expensive in gas terms. Though, we will explore options for checking against all collateral in the patches."

Fix Review:

Fixed.

Low Severity Issues

L-01: OKX Plugin _extractFromTokenCommissionRate can read calldata at out-of-bounds offsets for multi-referrer commission layouts

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/plugins/swap/OKXPlugin.sol	Status: Fixed	

Description:

When referrerNum >= 2, _extractFromTokenCommissionRate() reads additional commission words backwards from the end of swapData. The current swapData.length >= 0x44 guard is only sufficient for the last commission word and the helper/token word, not for all extra backwards reads. For short payloads, the computed read offset can move before the start of the swapData, causing calldataload to consume unrelated ABI bytes and derive totalRate from unintended data.

Recommendations:

Add a length check before the loop ensuring swapData.length >= 0x24 + referrerNum * 0x20.

Customer Response:

Fixed in commit f5f37df .

Fix Review:

Fixed.

L-02: Swap adapters do not enforce full amountIn consumption, so partial fills finalize as if the whole operation executed

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/CometFoundation.sol contracts/plugins/swap/**	Status: Acknowledged	

Description:

`CometFoundation` treats any unspent source token left after `_swap()` as dust instead of enforcing that the full requested `amountIn` was actually consumed. In `multiply()` and `adjust()`, this means a swap may succeed with `amountOut >= minReturn` while consuming less than the intended source amount, after which the leftover source token is refunded to the user and the protocol still borrows the full `repaymentAmount` to settle the flash loan. As a result, leverage-up flows can finalize with less collateral added, weakening the health factor of the user more and potentially leaving the user with a riskier position than intended.

Recommendations:

Consider enforcing slippage on source-token consumption in all plugins. Compare pre/post source-token balances and ensure the actual spend matches the requested amount.

Customer Response:

Acknowledged - *"The health factor is now validated after the operation, thus, the HF shouldn't drop unexpectedly for user and revert if drop exceeds health buffer."*

Fix Review:

Acknowledged.

L-03: Front-runnable allowBySig enables denial of service (DoS) on user transactions

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/CometFoundation.sol	Status: Fixed	

Description:

The _allow function unconditionally calls comet.allowBySig() with the user's EIP-712 signature. An attacker can monitor the mempool and directly call comet.allowBySig using the intercepted signature. This permanently consumes the user's nonce. When the user's legitimate transaction processes, it reverts due to a consumed signature, allowing systematic griefing.

Recommendations:

Before attempting to call comet.allowBySig, verify if the foundation already holds the required permissions by querying comet.isAllowed(msg.sender, address(this)). Skip the signature consumption if the allowance is already granted.

Customer Response:

Fixed in commit 11a41b4 .

Fix Review:

Fixed.

L-04: Adjusted event reverts on underflow when user has base supply

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/CometFoundation.sol	Status: Fixed	

Description:

In _process, debt is read from comet.borrowBalanceOf(user) after the withdrawFrom call. The previousDebt argument is then computed as debt - repaymentAmount. This assumes withdrawFrom increased the borrow balance by exactly repaymentAmount, but Comet first drains any existing base supply before creating debt. If the user has positive base supply, withdrawFrom can consume some or all of that supply before creating debt. When the resulting borrowBalanceOf(user) is lower than repaymentAmount, debt - repaymentAmount underflows and reverts.

```
uint256 debt = comet.borrowBalanceOf(user);
emit ICE.Adjusted(
    user,
    address(comet),
    address(params.supplyAsset),
    debt - repaymentAmount,
    debt,
    amountOut
);
```

Recommendations:

Read the borrow balance before any state changing interaction so the previousDebt value will reflect the actual pre interaction state.

Customer Response:

Fixed in commit 99dd88c .

Fix Review:

Fixed.

Informational Severity Issues

I-01: Swap plugins and external calls fail to reset approvals, accumulating residual allowances

Files:

contracts/CometFoundation.sol

Description:

Swap plugins and the CometFoundation repeatedly use safeIncreaseAllowance to grant token approvals to DEX routers, approve proxies, and flash loan providers. If the provider does not fully consume the approved amount, the residual allowance permanently remains assigned to them. Over time, these outstanding allowances can expose stuck or newly accumulated tokens in the foundation to unintended spending if an approved third-party integration is compromised.

Recommendations:

Enforce safe allowance management by strictly resetting external allowances to zero after an operation completes. Use SafeERC20.forceApprove(spender, 0) immediately after executing the external call.

Customer Response:

Fixed in commit da2e533 .

Fix Review:

Fixed.

I-02: adjust() is functionally redundant with multiply() and can be merged

Files:

contracts/CometFoundation.sol

Description:

adjust() increases leverage on an existing position without depositing new collateral. It is semantically identical to multiply() with collateralAmount = 0. The only real difference between the two functions is their leverage check logic — and notably, _adjust() has the correct one (after M-01's fix): it anchors to the user's full existing collateral balance and accounts for existing debt, while _multiply() ignores both. A user calling adjust(debtDelta) and a user calling multiply(collateral, 1 wei, debtDelta, ...) end up in the same on-chain state.

Maintaining two separate code paths for the same operation unnecessarily expands the attack surface and adds a divergent Mode.ADJUST branch, a separate ICometAdjust interface, and a separate Adjusted event.

Recommendations:

Remove adjust(). Modify _multiply() to skip transferFrom when collateralAmount == 0, and fix its leverage check to account for the user's existing collateral and borrow balance (or do a post tx check) — the correct formula already exists verbatim in _adjust(). Remove Mode.ADJUST, ICometAdjust, and the corresponding branch in _process().

Customer Response:

Fixed in commit 99dd88c .

Fix Review:

Fixed.

I-03: UniswapV3Plugin rejects zero-fee flash loans unnecessarily

Files:

contracts/plugins/flashloan/UniswapV3Plugin.sol

Description:

The uniswapV3FlashCallback function contains a strict check that the flash loan fee must be non-zero (fee != 0). This unnecessary check prevents the plugin from working with Uniswap V3 forks or pools that may offer zero-fee flash loans.

Recommendations:

Remove the fee != 0 check, keeping only the sentinel validation require(fee != type(uint256).max, ICA.InvalidFlashLoanData());.

Customer Response:

Fixed in commit 244fb08 .

Fix Review:

Fixed.

I-04: cover() function's strict loanDebt <= borrowBalance check prevents fully closing positions due to interest accrual

Files:

contracts/CometFoundation.sol

Description:

The cover function restricts loanDebt to be strictly less than or equal to the off-chain calculated borrowBalance. Because Compound V3's borrowBalance continuously accrues interest dynamically, the actual balance may have increased by the time the transaction is executed. The user will therefore repay slightly less than the full debt, leaving dust debt. Users are mathematically prevented from fully closing their positions in a single transaction.

Recommendations:

Consider updating _cover() to recognize type(uint256).max for loanDebt, resolving it to the execution-time borrowBalance to guarantee a clean unwind of the position.

Customer Response:

Fixed in commit a160366 .

Fix Review:

Fixed.

I-05: _multiply() does not validate collateralAmount > 0 for ERC-20 path

Files:

contracts/CometFoundation.sol

Description:

When executing `_multiply()` with `msg.value == 0`, the function fails to explicitly require that `collateralAmount > 0`. A value of zero passes initial stages but eventually fails deep in the path, wasting gas.

Recommendations:

Add an explicit `require(collateralAmount > 0, ICA.InvalidAmountIn());` check early in the ERC-20 execution path.

Customer Response:

Fixed in commit 9f7a4ca .

Fix Review:

Fixed.

I-06: _exchange() does not validate fromAmount against user's collateral balance

Files:

contracts/CometFoundation.sol

Description:

Unlike _cover(), the _exchange() function does not validate that fromAmount is less than or equal to the user's actual Comet collateral balance, causing it to revert late if the balance is insufficient.

Recommendations:

Add an early check that fromAmount <= comet.collateralBalanceOf(msg.sender, fromAsset) to fail fast on invalid input.

Customer Response:

Fixed in commit d9b01ac .

Fix Review:

Fixed.

I-07: Missing opts.comet validation allows arbitrary contract execution

Files:

contracts/CometFoundation.sol

Description:

The opts.comet parameter is fully caller-supplied, and the foundation only checks that it is non-zero before interacting with it. This means the contract assumes the caller passes a legitimate Compound V3 market. Restricting opts.comet to known market addresses would reduce the chance of interacting with an unintended contract through supplyTo, withdrawFrom, and related approval flows.

Recommendations:

Consider maintaining an immutable or admin-managed allowlist of approved Comet market addresses and require that opts.comet belongs to that set before executing market interactions.

Customer Response:

Acknowledged - *"Having allowlist of comets will require the redeployment of CometFoundation every time a new market should be added which creates deployment overhead. Also, we want to keep any data immutable, so that we do not need any admin setters to maintain smart contract. We will ensure validity of passed comets off-chain on the Frontend for users of the Compound UI."*

Fix Review:

Acknowledged.

I-08: Unvalidated swap routing parameters allow token diversion

Files:

contracts/plugins/swap/OKXPlugin.sol

contracts/plugins/swap/OneInchV6Plugin.sol

contracts/plugins/swap/LiFiPlugin.sol

Description:

Swap adapters such as 1inch, LiFi, and OKX forward user-supplied calldata to external routers after approving the input token, but they only validate a subset of the routing parameters on-chain. In particular, some execution-related fields remain unconstrained by the adapters themselves: for 1inch, srcReceiver and executor; for LiFi, approveTo and callTo; for OKX, parts of the UniV3 path and appended commission metadata. As a result, swap safety is only partially enforced on-chain, execution behavior and token flows is not enforced and still depends on the correctness of the frontend or quote provider supplying the calldata.

Recommendations:

Consider avoiding raw forwarding of aggregator payloads unless the relevant routing fields can be constrained and validated on-chain. Otherwise, document that these integrations rely on trusted input generation.

Customer Response:

Acknowledged - "Will document that execution relies on the correctness of swap data generated by the provider's API. Though, there are several on-chain validations implemented where possible such as: i) Validate tokens from quote correspond to tokens specified by users. ii) Validate amount in specified by user and amount in from quote. iii) Validate that the contract received a minReturn amount of toToken after swap. Documented it in commit a9c9d78."

Fix Review:

Acknowledged.

I-09: cover() lacks deleveraging invariant, allowing transactions to silently worsen account safety

Files:

contracts/CometFoundation.sol

Description:

The _cover() function does not strictly enforce that the operation actually improves the user's position (i.e., reduces leverage). If the withdrawn collateral has a high collateral factor or if the swap requires significantly more collateral to cover fees and slippage, the liquidity removed from the Comet position can exceed the debt repaid.

Recommendations:

Enforce a deleveraging invariant on-chain. Snapshot the user's pre-operation borrow "headroom" and ensure that the post-operation "headroom" is greater than or equal to the pre-operation one.

Customer Response:

Acknowledged - *"It might be too expensive in gas terms to validate on-chain. Will calculate and show the change in HF after the operation in FE."*

Fix Review:

Acknowledged.

I-10: Zero-value plugin selectors can be registered even though they are unusable at runtime

Files:

contracts/CometFoundation.sol

Description:

The constructor never enforces that a plugin must have a valid non-zero selector. A plugin returning bytes4(0) can be registered and will emit PluginAdded, but _config() later rejects zero selectors, creating dead registry entries.

Recommendations:

Validate pluginSelector != bytes4(0) during construction and reject malformed plugins up front.

Customer Response:

Acknowledged - *"Since data is immutable and only trusted implementations of plugins will be used, the validation will be performed before and after deployment."*

Fix Review:

Acknowledged.

I-11: _dust() NatSpec no longer matches the new dust-handling logic

Files:

contracts/CometFoundation.sol

Description:

The NatSpec for _dust() still says base-asset dust is transferred directly to the user when comet != address(0). The current implementation no longer special-cases the base asset and instead supplies any ERC20 dust to Comet whenever a non-zero comet address is passed.

```
/**
* @notice Handles any leftover tokens by either supplying to Comet or
transferring to the user
* @param user Address of the user to receive leftover tokens
* @param asset The ERC20 token to handle
* @param comet The Comet market instance (or address(0) if supply not needed)
* @param amount Amount of tokens to handle
* @dev If comet is address(0), tokens are always transferred to user.
* Otherwise, if asset is baseAsset, tokens are transferred; if collateral, they
are supplied to Comet.
*/
```

Recommendations:

Update the NatSpec to reflect the new behavior.

Customer Response:

Fixed in commit 7a71cdd .

Fix Review:

Fixed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.